

Embedded Systems Laboratory Report

BLE Central Programming with a Raspberry Pi

Basic Problem and Option Problem 1

YU-CHEN, CHU
Student ID: R12942157

October 7, 2026

Source code: https://github.com/oldpig123/embedded_system/tree/master/lab3

Contents

1	Introduction	2
2	Environment	2
2.1	Raspberry Pi setup	2
3	Basic Problem: CCCD Write from a Python Central	2
3.1	Design	2
3.2	Selecting the peripheral	3
3.3	Implementation	3
3.4	The peripheral used for testing	4
3.5	Results	5
4	Problems Encountered	6
5	Option Problem 1: BLE in C with GATTLIB	6
5.1	Building GATTLIB	6
5.2	Trying the examples	7
6	Conclusion	7

1 Introduction

This exercise is about the client side of Bluetooth Low Energy (BLE). A Raspberry Pi acts as a BLE *central*, which in GATT terms is the *client*, and connects to a *peripheral*, the GATT *server*. The task is to change the value of a Client Characteristic Configuration Descriptor (CCCD) on the server from the Python program running on the Pi.

The CCCD (UUID 0x2902) is the mechanism by which a client controls which server-initiated packets it receives for a given characteristic. A server may send a notification only if the client has written 0x0001 to the CCCD, and an indication only if it has written 0x0002. The value is a 16-bit little-endian integer, so the byte sequence actually sent over the air for 0x0002 is 02 00. This byte order is an easy mistake to make and is called out again in the implementation section.

The report covers the basic problem (a Python central that writes 0x0002 to a CCCD) and the optional problem (building the C library GATTLIB on the Pi and trying its examples).

2 Environment

Two machines are involved. The Raspberry Pi is the device under test and runs the central. A Windows laptop stands in for the phone and runs a small GATT server (see Section 3.4 for why).

Table 1: Software and hardware used.

	Raspberry Pi (central)	Windows laptop (peripheral)
OS	Debian 13 (“trixie”)	Windows 11
Bluetooth	Pi onboard adapter (hci0)	Qualcomm FastConnect 7800
BLE stack	BlueZ 5.82	Windows WinRT GATT API
Language	Python 3.13, C (gcc 14.2)	Python 3.14 (managed by uv)
Libraries	bluepy 1.3.0, GATTLIB	winrt-* 3.2.1

2.1 Raspberry Pi setup

The lecture notes install bluepy with `sudo pip install`. On Debian 13 this is refused with `error: externally-managed-environment` (PEP 668): the distribution prevents `pip` from writing into the system Python so that it cannot break OS packages. The fix is a virtual environment:

```
1 sudo apt install python3-pip libglib2.0-dev
2 python3 -m venv ~/blenv
3 ~/blenv/bin/pip install bluepy
4 sudo ~/blenv/bin/python central.py      # root is needed for BLE scanning
```

Listing 1: Installing bluepy on Debian 13.

Because the program must run as root, `sudo` has to be given the virtual environment’s interpreter explicitly. A plain `sudo python` would use the system Python, in which `bluepy` is not installed.

3 Basic Problem: CCCD Write from a Python Central

3.1 Design

The program follows the scan-connect-write sequence of the lecture notes:

1. Scan for 8 seconds and select the peripheral that advertises service 0xFF0.
2. Connect to it.
3. Find characteristic 0xFF1 and, from it, its CCCD (0x2902).
4. Write the two bytes 02 00 (the value 0x0002) with a write *request*, so that the server's response, or its error, is observed.
5. Read the descriptor back and print it, then disconnect in a **finally** block so that the link is released even if a step fails.

3.2 Selecting the peripheral

An unfiltered scan in a normal room returned about 70 devices, so the target must be identified by content rather than by position in the list. The peripheral's address is not usable for this: it is a *random* address and *changes* from one advertising session to the next (the same laptop appeared under several different addresses across runs). The program therefore matches on the advertised service UUID and then uses whatever address that entry carries, passing the address type 'random' to `Peripheral`. Without the address type, `bluepy` fails with "connection refused (111)", as noted in the lecture slides.

Two details of the advertisement matter. First, `getValueText()` returns `None` for entries that lack the requested field, which is true of most devices in a busy room, so the `None` case must be skipped before the substring test. Second, the target UUID was found under AD type 3 (complete list of 16-bit service UUIDs), not AD type 7 (128-bit), because the 0xFF0 UUID is a 16-bit short form of the Bluetooth base UUID. A first attempt that looked only at type 7 found nothing even though the device was being received.

3.3 Implementation

```

1  # Scan, then choose the entry where getValueText(3) contains fff0. A name match
   # on OLDPIG123_HOST also works.
2  # Peripheral(addr, 'random').
3  # Get the characteristic with getCharacteristics(uuid=UUID(0xff1))[0].
4  # Get its CCCD with getDescriptors(forUUID=0x2902)[0].
5  # Write b"\x02\x00" with withResponse=True.
6  # Read it back and print it.
7  # Disconnect in a finally.
8
9  from bluepy.btle import Scanner, Peripheral, UUID, BTLEException
10
11 scanner = Scanner()
12 print("Scanning for 8 seconds...")
13 scan_entries = scanner.scan(8.0)
14
15 for entry in scan_entries:
16     text = entry.getValueText(3)
17     # check text is not None before checking for substring
18     if text is None:
19         continue
20     if "fff0" in text:
21         print(f"Found device: {entry.addr}")
22         break
23 else:
24     print("No device found with UUID fff0.")
25     exit(1)
26
27 peripheral = Peripheral(entry.addr, 'random')
28 try:

```

```

29     # Get the characteristic with getCharacteristics(uuid=UUID(0xff1))[0].
30     char = peripheral.getCharacteristics(uuid=UUID(0xff1))[0]
31
32     print(char.propertiesToString()) # prints "INDICATE"
33     # Get its CCCD with getDescriptors(forUUID=0x2902)[0].
34     cccd = char.getDescriptors(forUUID=0x2902)[0]
35
36     print(f"CCCD handle: 0x{cccd.handle:04x}") # prints 0x000e
37     print("1 for notification, 2 for indication.")
38     # Write b"\x02\x00" with withResponse=True.
39     cccd.write(b"\x02\x00", withResponse=True)
40     # Read it back and print it.
41     value = cccd.read()
42     n = int.from_bytes(value, "little")
43     print(f"CCCD value: 0x{n:04x}") # prints 0x0002
44
45 finally:
46     # Disconnect in a finally.
47     peripheral.disconnect()

```

Listing 2: `central.py`: BLE central running on the Raspberry Pi.

The CCCD is located through the *characteristic* object (`char.getDescriptors(forUUID=0x2902)`), not through the peripheral: `Peripheral.getDescriptors` takes a handle range and has no `forUUID` argument. The value is written as `b"\x02\x00"`; the bytes `00 02` would instead set the value to `0x0200`.

3.4 The peripheral used for testing

The assignment suggests an Android phone running BLE Tool or a BLE scanner app. Installing an app on the phone was not desired, so the peripheral role was instead played by the Windows laptop. Its Bluetooth adapter was confirmed to support the BLE peripheral role (`IsPeripheralRoleSupported = True` through the WinRT BluetoothAdapter API). A short Python program, `server.py`, advertises service `0xFFFF0` containing one *indicate-only* characteristic `0xFFFF1`, and prints a timestamped line whenever a client's subscription changes.

```

1  # server.py: stand-in for the Android "BLE Tool" GATT server, run on the Windows
   # laptop.
2  # Advertises service 0xFFFF0 with one indicate-only characteristic 0xFFFF1 and
   # logs
3  # every time a central changes its subscription (i.e. writes the CCCD).
4  import asyncio
5  import time
6  import uuid
7
8  from winrt.windows.devices.bluetooth.genericattributeprofile import (
9      GattCharacteristicProperties,
10     GattLocalCharacteristicParameters,
11     GattServiceProvider,
12     GattServiceProviderAdvertisingParameters,
13 )
14
15 SERVICE_UUID = uuid.UUID("0000fff0-0000-1000-8000-00805f9b34fb")
16 CHAR_UUID = uuid.UUID("0000fff1-0000-1000-8000-00805f9b34fb")
17
18
19 def on_subscribed_clients_changed(sender, args):
20     # Windows owns the CCCD, so the raw 0x0001/0x0002 value is not visible here;
21     # we can only see who is subscribed. The characteristic is indicate-only, so
   # a
22     # subscription can only come from a CCCD write of 0x0002.
23     n = len(sender.subscribed_clients)

```

```

24     print(f"[{time.strftime('%H:%M:%S')}] CCCD changed on {CHAR_UUID}: {n}
      subscribed client(s)", flush=True)
25
26
27 async def main():
28     result = await GattServiceProvider.create_async(SERVICE_UUID)
29     provider = result.service_provider
30
31     params = GattLocalCharacteristicParameters()
32     params.characteristic_properties = GattCharacteristicProperties.INDICATE
33     char_result = await provider.service.create_characteristic_async(CHAR_UUID,
        params)
34     char_result.characteristic.add_subscribed_clients_changed(
        on_subscribed_clients_changed)
35
36     adv = GattServiceProviderAdvertisingParameters()
37     adv.is_discoverable = True
38     adv.is_connectable = True
39     provider.start_advertising_with_parameters(adv)
40     print(f"Advertising service {SERVICE_UUID}. Ctrl+C to stop.", flush=True)
41     try:
42         while True:
43             await asyncio.sleep(1)
44     finally:
45         provider.stop_advertising()
46
47
48 asyncio.run(main())

```

Listing 3: `server.py`: GATT server on the laptop, standing in for the phone.

The characteristic is deliberately indicate-only, so that the only valid CCCD value is 0x0002.

The choice of library was not the first one. The popular `bless` package was tried first, but version 0.2.6 does not work on current Windows: it imports the obsolete `bleak_winrt` module, requires a `pysetupdi` package that is not on PyPI, and, to set the device name, edits the adapter's registry entry. The server was therefore written directly against the WinRT `GattServiceProvider` classes, which needs only a few calls.

A limitation of this peripheral. On Windows the operating system owns the CCCD. Unlike the Android BLE Tool, whose server log shows the raw descriptor write, a Windows server program can observe only the `SubscribedClientsChanged` event, i.e. how many clients are subscribed, not the 0x0002 value itself. The evidence for the write is therefore the combination of two independent observations: the Pi reads the descriptor back as 0x0002, and the laptop's server log records the subscription. The indicate-only property is what ties a subscription to the value 0x0002.

3.5 Results

With `server.py` running on the laptop, `central.py` found the device, wrote the descriptor, and read back the value 0x0002. The laptop's log showed a subscription at the moment of the write:

```

1 Advertising service 0000fff0-0000-1000-8000-00805f9b34fb. Ctrl+C to stop.
2 [15:48:11] CCCD changed on 0000fff1-...: 1 subscribed client(s)
3 [15:48:13] CCCD changed on 0000fff1-...: 0 subscribed client(s)

```

Listing 4: Laptop log during one run of the central.

The second line is *not* a second write. It is the cleanup that occurs when the Pi disconnects in the `finally` block: a CCCD subscription does not survive the connection, so the server sees

the subscriber count return to zero. The two timestamps therefore show subscribe at the write and unsubscribe at the disconnect.

4 Problems Encountered

Most of the time in this lab was spent not on the CCCD logic itself but on getting a working BLE environment. The causes are listed because each one produces a misleading symptom.

Table 2: Problems, their symptoms and causes.

Problem	Symptom	Cause and fix
<code>pip install</code> refused	externally-managed-environment	Debian 13 enforces PEP 668. Use a venv and run it with <code>sudo</code> by absolute path.
Scan finds nothing, <code>scanend ...</code> Rejected	<code>bluepy</code> raised <code>BTLEManagementError</code> after the scan; empty device list; <code>bluetoothctl</code> said <code>NotReady</code>	Bluetooth was <i>soft-blocked</i> (<code>rfkill list</code>). Fixed with <code>rfkill unblock bluetooth</code> and <code>hciconfig hci0 up</code> . The error message pointed at <code>bluepy</code> , but the library was not at fault.
<code>bleess</code> fails on Windows	No module named <code>bleak_winrt</code> , then <code>pysetupdi</code>	Unmaintained dependencies. Replaced by direct WinRT calls.
Laptop invisible to the Pi	Filter on AD type 7 found nothing	The UUID is in AD type 3. A raw dump of all scan fields exposed it.
Address kept changing	A stored MAC no longer matched	Random advertising addresses rotate; select by service UUID instead.

The soft-blocked adapter was the most instructive case. Two unrelated tools, `bluepy` and `bluetoothctl`, failed with different messages, and only the lower-level `rfkill` state explained both. When a stack of libraries reports an error, the fault should be looked for at the lowest layer first.

5 Option Problem 1: BLE in C with GATTLIB

5.1 Building GATTLIB

GATTLIB was cloned from <https://github.com/labapart/gattlib> and built with CMake on the Pi. CMake selected the D-Bus backend for BlueZ 5.82. The build initially failed at configuration time, in the examples only, with None of the required 'libpcre' found.

The cause is that the examples' `CMakeLists.txt` files request the PCRE1 library (`libpcre`), while Debian 13 no longer ships `libpcre3-dev` (`apt-cache policy` reported no candidate). PCRE2 (`libpcre2-dev`) was already installed as a dependency of GLib. Before changing anything, a search of the C sources showed that none of the GATTLIB code actually calls the PCRE API; only the CMake files link it. The one-word change in eight files, from `libpcre` to `libpcre2-8`, was therefore enough:

```

1 sudo apt install git cmake build-essential libbluetooth-dev libreadline-dev
2 git clone https://github.com/labapart/gattlib.git && cd gattlib
3 grep -rl "REQUIRED libpcre)" --include=CMakeLists.txt . \
4 | xargs sed -i 's/REQUIRED libpcre)/REQUIRED libpcre2-8)/'
5 mkdir build && cd build && cmake .. && make

```

```
6 sudo make install
```

Listing 5: Build steps, including the PCRE workaround.

The build and installation completed. The examples were run from the `build` directory. The compiler emitted warnings (transposed `calloc` arguments, an unused variable) from the library’s own code under gcc 14, which do not affect the result.

5.2 Trying the examples

ble_scan. This example scans and then attempts to connect to *every* device it found, listing each one’s GATT services. In a room with several dozen random-address devices most connections fail, for example with `Timeout was reached` or `le-connection-abort-by-local`. These failures are expected rather than a fault in the setup: the devices are other people’s phones, trackers and peripherals that do not accept a connection from a stranger. One device did accept and its services (0x1800, 0x1801, 0x180A, 0x180F and several vendor-specific UUIDs) were listed, which demonstrates that scanning, connecting and service discovery all work. The program also terminated with a segmentation fault when it attempted to connect to a network printer. This crash was observed but not investigated; it occurs in the example’s failed-connection path, not in the code written for this lab.

When the output was piped to `tee` for logging, the log initially came out empty. This is standard C buffering: when standard output is a pipe it is block-buffered, and the process was interrupted with Ctrl+C before the buffer was flushed. Running it through `stdbuf -oL` (line-buffered) and redirecting standard error into the same pipe fixed it.

discover. Aimed at the laptop running `server.py`, `discover` listed the laptop’s complete GATT table, which includes many built-in Windows services, and ended with the characteristic written by `server.py`:

```
1 characteristic[36] properties:20 value_handle:0068 uuid:0xffff1
```

Listing 6: The last line of `discover`’s output.

The property byte 0x20 is the GATT *Indicate* bit, which matches how `server.py` defined the characteristic, so GATTLIB read the true table from the laptop. The `read_write` and `notification` examples need a readable characteristic and a Battery service respectively, and neither exists on the test peripheral.

6 Conclusion

A Python BLE central was written that scans, selects its peripheral by advertised service, connects, writes 0x0002 to a CCCD with a write request, reads the value back and disconnects cleanly. It was tested against a purpose-written GATT server on a Windows laptop instead of an Android phone. The principal limitation of that choice is that Windows hides the raw CCCD value from the server application, which is why the evidence combines the Pi’s read-back with the server’s subscription event.

In the optional part, GATTLIB was built on a Debian 13 system for which its documented dependencies are partly out of date, and two of its examples were used to scan and to enumerate the services of a device under test.

Beyond the CCCD itself, the main lesson was how much of BLE programming is about the environment: soft-blocked radios, address types and rotating addresses, byte order, and libraries that have not kept pace with the operating system. Each of those produced a symptom far from its cause, and each was solved by testing the layer below the one that reported the error.